

# **Путь Python-разработчика**

**От полного новичка до первой работы**

Владислав

8 августа 2026

# Содержание

---

|            |                                                               |           |
|------------|---------------------------------------------------------------|-----------|
| <b>1</b>   | <b>Путь Python-разработчика</b>                               | <b>5</b>  |
| <b>I</b>   | <b>2 Том I. Основы программирования</b>                       | <b>6</b>  |
|            | Быстрые ссылки . . . . .                                      | 7         |
| <b>2.1</b> | <b>Как думает компьютер</b>                                   | <b>8</b>  |
|            | После этой главы вы сможете . . . . .                         | 8         |
|            | Вы уже сталкивались с программированием . . . . .             | 9         |
|            | Задача, алгоритм и программа . . . . .                        | 10        |
|            | Компьютер не умеет догадываться . . . . .                     | 11        |
|            | Первая практика . . . . .                                     | 12        |
|            | Короткий итог . . . . .                                       | 12        |
| <b>2.2</b> | <b>Почему компьютер понимает только 0 и 1</b>                 | <b>14</b> |
|            | Почему компьютер не понимает человеческий язык . . . . .      | 14        |
|            | Что находится внутри компьютера . . . . .                     | 15        |
|            | Почему именно два состояния . . . . .                         | 17        |
|            | Откуда появились 0 и 1 . . . . .                              | 18        |
|            | Что такое бит . . . . .                                       | 20        |
|            | Как из двух состояний получается сложная информация . . . . . | 21        |
|            | Всё превращается в числа . . . . .                            | 22        |
|            | Что происходит технически . . . . .                           | 24        |
|            | От транзисторов к логическим операциям . . . . .              | 25        |
|            | Немного истории . . . . .                                     | 26        |
|            | Почему это важно будущему Python-разработчику . . . . .       | 27        |
|            | Практика . . . . .                                            | 28        |
|            | Задание 1 . . . . .                                           | 28        |
|            | Задание 2 . . . . .                                           | 28        |
|            | Задание 3 . . . . .                                           | 29        |
|            | Ответы к практике . . . . .                                   | 29        |
|            | Ответ к заданию 1 . . . . .                                   | 29        |
|            | Ответ к заданию 2 . . . . .                                   | 30        |
|            | Ответ к заданию 3 . . . . .                                   | 30        |
|            | Проверьте себя . . . . .                                      | 31        |

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| Ответы для самопроверки . . . . .                               | 31        |
| Главное, что нужно запомнить . . . . .                          | 32        |
| Что дальше . . . . .                                            | 33        |
| <b>2.3 Как компьютер хранит данные</b>                          | <b>34</b> |
| Бит — самая маленькая единица информации                        | 35        |
| Байт                                                            | 36        |
| Как компьютер понимает, что хранится в байте?                   | 38        |
| Как хранится текст                                              | 40        |
| Почему этого оказалось недостаточно                             | 42        |
| Как хранится число                                              | 43        |
| Как хранится изображение                                        | 44        |
| Как хранится музыка                                             | 46        |
| Как хранится программа                                          | 47        |
| Главное, что нужно запомнить                                    | 49        |
| Что дальше?                                                     | 50        |
| <b>2.4 Как компьютер выполняет программы</b>                    | <b>51</b> |
| После этой главы вы сможете . . . . .                           | 51        |
| Программа — это инструкция для компьютера . . . . .             | 51        |
| Где находится программа до запуска . . . . .                    | 52        |
| Что происходит после запуска . . . . .                          | 52        |
| Как процессор выполняет инструкции . . . . .                    | 53        |
| Откуда процессор знает, какую инструкцию выполнять . . . . .    | 54        |
| А где здесь Python? . . . . .                                   | 55        |
| Короткий итог . . . . .                                         | 56        |
| Практика . . . . .                                              | 56        |
| <b>2 2.5 Почему Python</b>                                      | <b>57</b> |
| 2.1 Можно ли писать программы сразу на машинном языке . . . . . | 57        |
| 2.2 Язык программирования нужен человеку . . . . .              | 58        |
| 2.3 Почему языков программирования так много . . . . .          | 60        |
| 2.4 Почему для обучения выбран Python . . . . .                 | 61        |
| 2.5 Python не лучший язык для всего . . . . .                   | 63        |
| 2.6 В чём сила Python . . . . .                                 | 64        |
| 2.7 Где используется Python . . . . .                           | 64        |
| 2.8 Веб-разработка . . . . .                                    | 65        |
| 2.9 Автоматизация . . . . .                                     | 65        |

---

|                                                   |           |
|---------------------------------------------------|-----------|
| 2.10 Анализ данных . . . . .                      | 66        |
| 2.11 Машинное обучение . . . . .                  | 66        |
| 2.12 Тестирование . . . . .                       | 66        |
| 2.13 Простота не означает примитивность . . . . . | 66        |
| 2.14 Что именно мы будем изучать . . . . .        | 67        |
| 2.15 Что важно запомнить . . . . .                | 67        |
| 2.16 Что дальше? . . . . .                        | 68        |
| <b>2.6 Рабочее пространство Python . . . . .</b>  | <b>69</b> |
| После этой главы вы сможете . . . . .             | 69        |
| Основные инструменты . . . . .                    | 69        |
| Где скачать Python . . . . .                      | 70        |
| Python — это интерпретатор . . . . .              | 71        |
| Что такое редактор кода . . . . .                 | 72        |
| Структура рабочего проекта . . . . .              | 72        |
| Терминал . . . . .                                | 73        |
| Виртуальное окружение . . . . .                   | 73        |
| Короткий итог . . . . .                           | 73        |
| Практика . . . . .                                | 74        |
| <b>2.7 Первая программа на Python . . . . .</b>   | <b>75</b> |
| Создаём файл программы . . . . .                  | 75        |
| Пишем первую строку Python . . . . .              | 76        |
| Что делает print . . . . .                        | 77        |
| Запускаем программу . . . . .                     | 78        |
| Что произошло после команды запуска . . . . .     | 79        |
| Код и результат — не одно и то же . . . . .       | 80        |
| Добавим ещё одну инструкцию . . . . .             | 80        |
| Порядок инструкций имеет значение . . . . .       | 81        |
| Программа должна быть записана точно . . . . .    | 82        |
| Ошибка — часть программирования . . . . .         | 83        |
| Читайте сообщения об ошибках . . . . .            | 84        |
| Первый маленький эксперимент . . . . .            | 85        |
| Практика: программа-визитка . . . . .             | 85        |
| Не бойтесь экспериментировать . . . . .           | 86        |
| Что мы уже умеем . . . . .                        | 87        |
| Что важно запомнить . . . . .                     | 87        |
| Что дальше? . . . . .                             | 88        |

# 1 Путь Python-разработчика

---

*От полного новичка до первой работы*

Практическая книга по Python и backend-разработке: от первых шагов до уверенной подготовки к первой работе.

Эта книга строится вокруг простого принципа: сначала мышление разработчика, затем синтаксис, инструменты и реальные проекты.

Вы будете постепенно разбирать задачи, превращать их в алгоритмы, писать понятный Python-код и собирать профессиональную рабочую среду.

## **i** Как читать книгу

Двигайтесь по главам последовательно. В начале важнее понимать ход мысли, чем запоминать команды.

## **Часть I**

# **2 Том I. Основы программирования**

В этом томе собраны базовые главы, которые помогают понять, как компьютер работает с инструкциями, данными и первыми программами.

## Быстрые ссылки

- [2.1 Как думает компьютер](#)
- [2.2 Почему компьютер понимает только 0 и 1](#)
- [2.3 Как компьютер хранит данные](#)
- [2.4 Как компьютер выполняет программы](#)
- [2.5 Почему Python](#)
- [2.6 Рабочее пространство Python](#)
- [2.7 Первая Python-программа](#)

### **i** Как читать этот том

Идите по главам последовательно. Сначала разберитесь, как компьютер воспринимает инструкции и данные, затем переходите к рабочему пространству и первой программе на Python.

## 2.1 Как думает компьютер

---

Прежде чем изучать переменные, условия, циклы и функции, полезно разобраться с более фундаментальным вопросом:

**что вообще происходит, когда человек пишет программу?**

На первый взгляд кажется, что программирование начинается с изучения языка. Например, с Python.

Но язык — это только инструмент.

Настоящее программирование начинается раньше: с умения сформулировать задачу и разбить её на последовательность понятных действий.

В этой главе мы разберёмся, что такое программа, почему компьютер не умеет догадываться, зачем существуют языки программирования и какую роль во всём этом играет Python.

### После этой главы вы сможете

После изучения главы вы сможете:

- объяснить своими словами, что такое программа;
- понимать разницу между задачей, алгоритмом и кодом;
- объяснить, почему компьютеру нужны точные инструкции;
- понимать, зачем существуют языки программирования;
- объяснить на базовом уровне, что происходит после запуска Python-программы;
- разбивать простую бытовую задачу на последовательность шагов.

#### **i** Главная цель главы

Пока не нужно запоминать команды Python.

Сначала необходимо понять принцип: **компьютер выполняет инструкции, а разработчик эти инструкции проектирует.**



## Вы уже сталкивались с программированием

Представим простую задачу:

**приготовить чашку чая.**

Для человека этой фразы обычно достаточно. Мы автоматически понимаем, что нужно сделать.

Налить воду. Включить чайник. Подготовить кружку. Положить чай. Дождаться кипения воды. Налить воду в кружку.

Большую часть этих действий мы даже не проговариваем.

Наш мозг самостоятельно заполняет пропущенные шаги.

Но теперь представим устройство, которое умеет выполнять команды, но совершенно не умеет догадываться.

Мы говорим ему:

Приготовь чай.

Для человека инструкция понятна.

Для машины — практически бесполезна.

Нужно описать задачу гораздо точнее:

1. Найди чайник.
2. Проверь, есть ли в нём вода.
3. Если воды недостаточно — добавь её.
4. Подключи чайник к электропитанию.
5. Включи чайник.
6. Возьми кружку.
7. Положи в неё чай.
8. Дождись кипения воды.
9. Налей горячую воду в кружку.
10. Подожди несколько минут.

Теперь большая задача превратилась в последовательность небольших действий.

Именно с этого начинается программирование.

### ! Запомните

**Программирование — это прежде всего умение разбивать задачу на понятные и точные шаги.**

Язык программирования нужен уже после того, как мы понимаем, какие действия должен выполнить компьютер.

## Задача, алгоритм и программа

На этом этапе важно различать три понятия:

**задача** — результат, который мы хотим получить;

**алгоритм** — последовательность действий для достижения результата;

**программа** — алгоритм, записанный в форме, которую компьютер способен выполнить.

Возьмём простой пример.

Наша задача:

узнать остаток денег после ежемесячных расходов.

Алгоритм можно описать обычным языком:

1. Узнать месячный доход.
2. Узнать месячные расходы.
3. Вычесть расходы из дохода.
4. Показать результат.

А позже мы сможем записать этот алгоритм на Python:

```
monthly_income = 5000
monthly_expenses = 3200

balance = monthly_income - monthly_expenses

print(balance)
```

Сейчас не нужно разбираться в каждой строке.

Важно увидеть связь:

```
Задача
  ↓
Алгоритм
  ↓
Код
  ↓
Выполнение
  ↓
Результат
```

Код не появляется из ниоткуда.

Перед кодом почти всегда должно существовать понимание того, **что именно должна делать программа.**

## Компьютер не умеет догадываться

Одна из важнейших мыслей для начинающего разработчика:

**компьютер не понимает намерения программиста.**

Он выполняет то, что написано.

Не то, что мы хотели написать.

Не то, что имели в виду.

И не то, что кажется очевидным человеку.

Представим инструкцию:

Если пользователь взрослый, разрешить регистрацию.

Человек понимает общий смысл.

Но компьютеру этого недостаточно.

Что значит «взрослый»?

18 лет?

21 год?

Возраст должен быть больше 18 или больше либо равен 18?

Что делать, если возраст вообще не указан?

Что делать, если вместо возраста пользователь ввёл слово?

Каждый подобный вопрос разработчику приходится превращать в конкретные правила.

Например:

```
Если возраст пользователя больше или равен 18:  
    разрешить регистрацию.  
Иначе:  
    отказать в регистрации.
```

Позже это превратится в настоящий Python-код.

💡 Как думает разработчик

Начинающий программист часто спрашивает:

Какую команду Python мне здесь написать?

Более полезный вопрос:

Какие точные действия должна выполнить программа?

Сначала решение. Затем код.

## Первая практика

Пока не открывайте Python.

Попробуйте выполнить упражнение обычным текстом.

Нужно написать инструкцию для робота, который должен почистить зубы.

Плохой вариант:

1. Взять щётку.
2. Почистить зубы.

Слишком много действий скрыто внутри второго шага.

Попробуйте описать процесс подробнее.

Например, подумайте:

- откуда взять зубную щётку;
- что сделать с зубной пастой;
- сколько пасты использовать;
- когда открыть воду;
- когда её закрыть;
- сколько времени чистить зубы;
- что сделать со щёткой после использования.

Чем точнее инструкция, тем ближе она к алгоритму.

### Типичная ошибка новичка

Не пытайтесь сразу переводить любую задачу в Python.

Сначала научитесь формулировать решение обычными словами.

Если алгоритм непонятен на русском языке, код почти наверняка тоже получится запутанным.

## Короткий итог

На этом этапе достаточно запомнить четыре вещи:

1. У программы всегда есть задача.

2. Задачу необходимо разбить на последовательность действий.
3. Компьютер выполняет инструкции буквально.
4. Python нужен для записи этих инструкций в форме, которую можно выполнить.

В следующем разделе мы разберёмся, **почему компьютер вообще способен работать только с очень простыми сигналами и откуда появились знаменитые 0 и 1.**

## 2.2 Почему компьютер понимает только 0 и 1

---

В предыдущей главе мы разобрались, что компьютер выполняет только точные инструкции.

Но возникает следующий вопрос:

**почему компьютер вообще работает с нулями и единицами?**

Почему нельзя заставить его сразу понимать слова, числа или команды так, как это делает человек?

Чтобы ответить на этот вопрос, нужно немного заглянуть внутрь компьютера.

Не слишком глубоко. Нам не понадобится изучать электронику или физику полупроводников.

Для Python-разработчика важно понять только основной принцип.

### Почему компьютер не понимает человеческий язык

Человек умеет использовать контекст.

Если сказать:

Сделай чай.

другой человек обычно понимает, что нужно сделать.

Он знает:

- где находится чайник;
- сколько примерно налить воды;
- какую кружку взять;
- когда вода закипела;
- что означает слово «чай».

Большая часть информации вообще не проговаривается.

Наш мозг автоматически достраивает недостающие шаги.

Компьютер так не умеет.

Если программа должна выполнить определённое действие, это действие необходимо описать достаточно точно.

Компьютер не понимает намерение разработчика.

Он выполняет только то, что действительно указано в программе.

### ! Главная мысль

Компьютер не обладает здравым смыслом.

Он не догадывается, что программист хотел сделать.

Он выполняет конкретные инструкции.

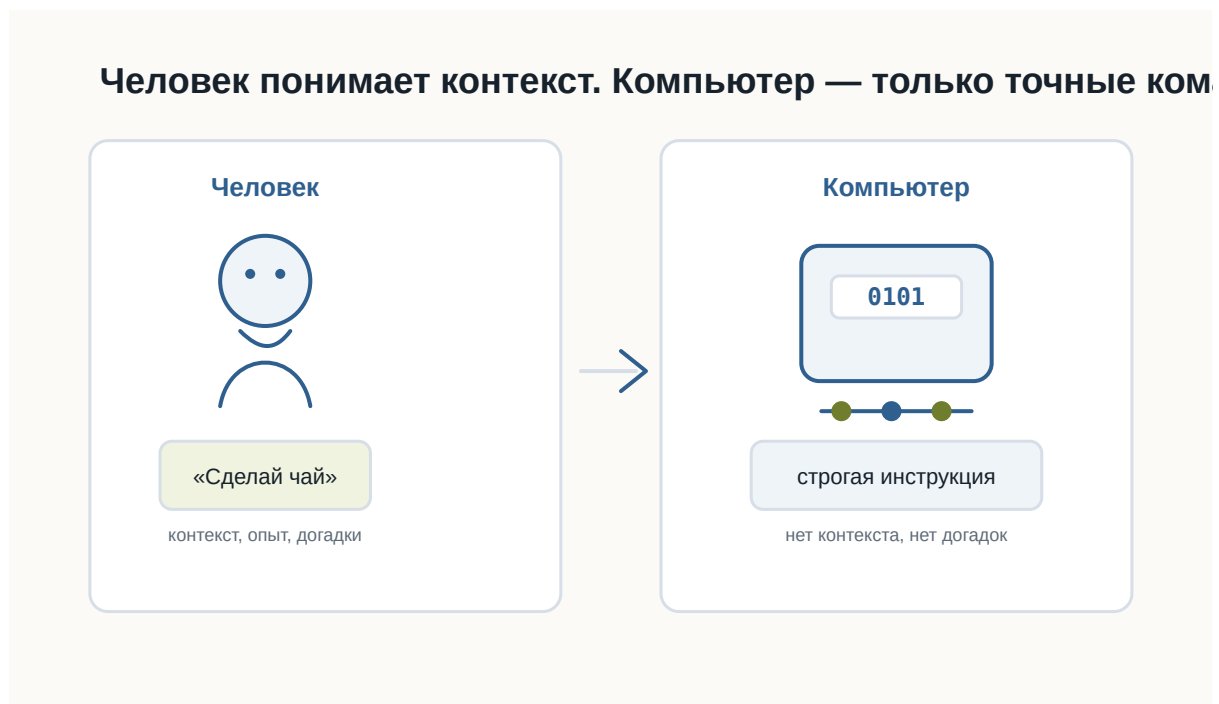


Рисунок 1.1: Человек использует контекст, а компьютеру нужны точные инструкции

## Что находится внутри компьютера

Современный компьютер состоит из огромного количества электронных компонентов.

Один из самых важных — **транзистор**.

Транзистор — это очень маленький электронный переключатель.

Для понимания программирования его можно представить как обычный выключатель света.

Выключатель имеет два устойчивых состояния:

Включён

Выключен

Транзистор работает по похожему принципу.

В упрощённом виде для нас достаточно считать, что он различает два состояния электрического сигнала:

Высокий уровень сигнала

Низкий уровень сигнала

Или ещё проще:

Есть сигнал

Нет сигнала

**i** Насколько точно это объяснение?

В реальной электронике всё немного сложнее.

Компьютер работает не с буквальными состояниями «ток есть» и «тока нет», а с диапазонами напряжений, которые электронные схемы интерпретируют как два логических состояния.

Но для первого знакомства модель «включено / выключено» достаточно точно передаёт основной принцип.

Современный процессор содержит огромное количество транзисторов.

Каждый из них очень мал, но вместе они способны выполнять миллиарды операций.

Чем больше транзисторов можно разместить внутри микросхемы и чем быстрее они переключаются, тем более сложные вычисления способен выполнять процессор.





Рисунок 1.2: Транзистор как электронный переключатель

## Почему именно два состояния

Можно задать вполне разумный вопрос:

Почему только два?

Почему не три?

Почему не десять?

Теоретически вычислительную систему можно построить и на другом количестве состояний.

Такие идеи действительно существовали и существуют.

Но двоичная система оказалась чрезвычайно удобной для электронной техники.

Причина — **надёжность**.

Представим, что электронная схема должна различать десять уровней напряжения.

Условно:

0.5 В  
1.0 В  
1.5 В  
2.0 В  
...

Тогда даже небольшая помеха может привести к ошибке.

Например, устройство ожидало 1.5 В, а из-за электрического шума получило 1.6 В.

Какое это состояние?

Чем больше состояний должна различать система, тем сложнее надёжно определить каждое из них.

С двумя состояниями гораздо проще.

Электронная схема может работать по принципу:

Низкий уровень → одно состояние

Высокий уровень → другое состояние

Между ними можно оставить запас.

Если сигнал немного изменится из-за помех, схема всё равно правильно определит его состояние.

Именно поэтому два состояния особенно хорошо подходят для быстрых и надёжных цифровых устройств.

#### 💡 Простая аналогия

Представьте два выключателя.

Первый имеет только два положения:

ВКЛ

ВЫКЛ

Второй имеет десять очень близко расположенных положений.

Каким выключателем проще пользоваться без ошибки?

Примерно по той же причине электронным схемам удобно работать с двумя состояниями.

## Откуда появились 0 и 1

Сами транзисторы не знают никаких цифр.

Внутри компьютера не бегают настоящие нули и единицы.

Цифры **0** и **1** придумали люди как удобные обозначения двух состояний.

Условно можно записать:

Низкий уровень сигнала → 0

Высокий уровень сигнала → 1

Можно было выбрать и другие обозначения:

НЕТ / ДА

ВЫКЛ / ВКЛ

FALSE / TRUE

ЧЁРНЫЙ / БЕЛЫЙ

Принцип работы компьютера от этого бы не изменился.

Но обозначения 0 и 1 оказались особенно удобными, потому что позволяют использовать **двоичную систему счисления**.

О ней мы поговорим подробнее позже.

Пока достаточно понимать:

**0 и 1 — это математические обозначения двух физических состояний.**

### Восемь включателей можно записать как восемь битов



Рисунок 1.3: Включённые и выключенные состояния можно записать как последовательность нулей и единиц

## Что такое бит

Теперь можно аккуратно познакомиться с первым важным термином.

**Бит** — это минимальная единица цифровой информации, которая может принимать одно из двух значений:

0

или

1

Слово bit произошло от английского выражения:

**binary digit** — двоичная цифра.

Один бит способен хранить только два варианта.

Например:

0 → нет

1 → да

или:

0 → свет выключен

1 → свет включён

Но одного бита недостаточно для хранения сложной информации.

Поэтому компьютеры используют большие последовательности битов.

Например:

01000001

Здесь уже восемь битов.

Группа из восьми битов называется **байтом**.

Подробно с битами, байтами и тем, как из них складываются данные, мы познакомимся в следующей главе.

**i** Не нужно зубрить

Пока достаточно запомнить:

- бит принимает значение 0 или 1;
- несколько битов можно объединять;
- из больших последовательностей битов компьютер строит сложные данные.

## Как из двух состояний получается сложная информация

На первый взгляд кажется странным:

как всего два состояния могут описать фотографию, музыку или целую компьютерную игру?

Ответ заключается в количестве комбинаций.

Возьмём один бит:

0  
1

Возможно только **2 комбинации**.

Теперь два бита:

00  
01  
10  
11

Уже **4 комбинации**.

Три бита:

000  
001  
010  
011  
100  
101  
110  
111

Получаем **8 комбинаций**.

Чем больше битов, тем больше разных состояний можно закодировать.

Для  $n$  битов количество комбинаций равно:

$$2^n$$

Например:

8 бит → 256 комбинаций

Именно поэтому даже небольшая последовательность битов уже может представлять достаточно много разных значений.

Компьютер использует огромные последовательности битов.

Поэтому двух базовых состояний оказывается достаточно для хранения практически любой цифровой информации.

## Всё превращается в числа

Компьютер не хранит букву так, как её видит человек.

Возьмём:

A

Компьютеру нужно договориться, какое число будет обозначать эту букву.

Например, в одной из распространённых систем кодирования латинской букве A соответствует число:

65

Число 65, в свою очередь, можно представить в двоичной форме:

01000001

Получается цепочка:

A

↓

65

↓

01000001

С изображениями идея похожая.

Фотография состоит из пикселей.

Каждый пиксель описывается числами.

Например, числа могут определять интенсивность красного, зелёного и синего цветов.

Дальше эти числа снова представляются последовательностями битов.

Фотография

↓

Пиксели

↓

Числа

↓

0 и 1

Музыка также может быть представлена числами.

Звуковая волна измеряется много раз в секунду.

Результаты измерений записываются как числа.

Звук

↓

Измерения

↓

Числа

↓

0 и 1

Видео можно рассматривать как последовательность изображений вместе со звуком.

Программы тоже состоят из данных и инструкций, которые в конечном итоге представлены тем же способом.



Рисунок 1.4: Текст, изображения, звук и программы в конечном итоге превращаются в цифровые данные

### ! Главная мысль

Компьютер способен работать с текстом, фотографиями, музыкой и программами не потому, что понимает их смысл. Он умеет представлять всё это числами. А числа можно представить последовательностями 0 и 1.

## Что происходит технически

Теперь немного точнее посмотрим на путь информации.

Когда программа работает с некоторым значением, происходит несколько уровней преобразований.

Упрощённо:

Данные программы  
↓  
Числовое представление  
↓  
Биты  
↓  
Электрические сигналы  
↓  
Электронные схемы



Например, Python-программа может работать со строкой:

```
message = "Hello"
```

Для разработчика это текст.

Python рассматривает его как объект строки.

Дальше компьютерное оборудование работает уже с машинным представлением этих данных.

В памяти находятся не слова в человеческом смысле, а закодированные значения.

На самом нижнем уровне они физически представлены состояниями электронных элементов.

Важно понимать, что программист почти никогда не работает напрямую с этим уровнем.

Python специально скрывает большую часть сложности.

Мы можем писать:

```
message = "Hello"
```

и не думать о том, какие конкретно биты находятся в оперативной памяти.

Это один из главных принципов программирования:

**каждый следующий уровень скрывает сложность предыдущего.**

Мы ещё много раз встретим эту идею в книге.

## От транзисторов к логическим операциям

Один транзистор сам по себе умеет немного.

Но транзисторы объединяют в электронные схемы.

Из них можно создавать так называемые **логические элементы**.

Они выполняют простые операции над двумя состояниями.

Например:

```
И  
ИЛИ  
НЕ
```

Позже из таких простых операций строятся более сложные схемы:

логические элементы



сумматоры



регистры



память



процессор

В результате миллиарды очень простых переключателей способны совместно выполнять сложные программы.

#### **i** Почему это важно программисту

Python-разработчику не нужно уметь проектировать процессоры. Но полезно понимать, что любой сложный код в конечном итоге выполняется системой, построенной из огромного количества простых логических операций. Это помогает избавиться от ощущения, что компьютер работает благодаря какой-то магии.

## Немного истории

Первые электронные компьютеры выглядели совсем не так, как современные ноутбуки.

До появления транзисторов в вычислительной технике широко использовались **электронные лампы**.

Они выполняли похожую функцию — позволяли управлять электрическими сигналами.

Но у них были серьёзные недостатки.

Лампы были:

- большими;
- горячими;
- энергозатратными;
- сравнительно ненадёжными.

Ранние компьютеры могли занимать целые помещения.

Появление транзистора позволило резко уменьшить размеры электронных схем.

Транзисторы были значительно меньше, экономичнее и надёжнее ламп.

Позже инженеры научились размещать множество транзисторов на одной микросхеме.

Количество транзисторов продолжало расти.

Сегодня внутри одного современного процессора могут находиться **миллиарды транзисторов**.

То, для чего когда-то требовалось целое помещение, теперь помещается в небольшом устройстве.



Рисунок 1.5: Упрощённая эволюция вычислительной техники от электронных ламп до современных процессоров

## Почему это важно будущему Python-разработчику

Можно задать вопрос:

Если Python скрывает все эти детали, зачем мне вообще знать про транзисторы и биты?

Чтобы писать первые программы, эти знания действительно не обязательны.

Но они помогают сформировать правильное представление о компьютере.

Когда позже мы начнём говорить о:

- переменных;
- типах данных;

- памяти;
- файлах;
- кодировках;
- базах данных;
- сетях;

вы будете понимать, что всё это разные способы организации одной и той же цифровой информации.

Кроме того, становится понятнее важный принцип:

**абстракции позволяют программисту работать со сложной системой, не думая постоянно о её нижних уровнях.**

Python — один из таких уровней абстракции.

## Практика

### Задание 1

Представьте четыре выключателя.

Каждый может быть либо включён, либо выключен.

Попробуйте самостоятельно записать все возможные комбинации из четырёх состояний с помощью 0 и 1.

Подсказка:

```
0000
0001
0010
...
```

Сколько комбинаций получится?

### Задание 2

Попробуйте объяснить своими словами:

Почему два состояния удобнее для электронной системы, чем десять?

Не используйте определение из книги дословно.

Главная задача — сформулировать мысль самостоятельно.

### Задание 3

Выберите любой объект:

- фотографию;
- песню;
- текстовый документ;
- видео.

Попробуйте мысленно пройти путь:

Объект

↓

Как его можно представить числами?

↓

Как числа можно представить битами?

Не нужно знать точные форматы файлов.

Нас интересует только общий принцип.

## Ответы к практике

Используйте этот раздел для самопроверки после того, как попробовали выполнить задания самостоятельно.

### Ответ к заданию 1

Для четырёх выключателей получится **16 комбинаций**:

```
0000
0001
0010
0011
0100
0101
0110
0111
1000
1001
1010
1011
1100
1101
```

1110  
1111

Почему именно 16:

$$2 \times 2 \times 2 \times 2 = 16$$

У каждого выключателя два состояния, а выключателей четыре.

## Ответ к заданию 2

Два состояния удобнее, потому что электронной системе проще быстро и надёжно отличать одно состояние от другого.

Например:

- есть сигнал или нет сигнала;
- высокое напряжение или низкое напряжение;
- включено или выключено.

Если состояний десять, системе пришлось бы гораздо точнее измерять сигнал и чаще решать, к какому состоянию он относится.

Это увеличивает риск ошибок.

## Ответ к заданию 3

Пример с текстовым документом:

Текстовый документ  
↓  
Символы получают числовые коды  
↓  
Числа записываются битами

Пример с фотографией:

Фотография  
↓  
Изображение разбивается на пиксели  
↓  
Цвет каждого пикселя записывается числами  
↓  
Числа записываются битами

Главное: объект не хранится в компьютере «как есть». Он переводится в числовое представление, а числа уже можно записать последовательностями 0 и 1.

## Проверьте себя

Попробуйте ответить без подсказок:

1. Что такое транзистор в упрощённом представлении?
2. Почему электронным схемам удобно использовать два состояния?
3. Действительно ли внутри компьютера находятся цифры 0 и 1?
4. Что такое бит?
5. Сколько возможных состояний имеет один бит?
6. Почему несколько битов позволяют хранить больше информации?
7. Как текст можно превратить в двоичные данные?
8. Можно ли по одному принципу хранить текст, музыку и фотографии?
9. Зачем Python-разработчику понимать эти основы, если Python скрывает работу электроники?

Если вы можете уверенно объяснить ответы своими словами, значит основная идея главы усвоена.

## Ответы для самопроверки

1. В упрощённом представлении транзистор можно считать маленьким электронным выключателем.

Он находится в одном из двух состояний: сигнал есть или сигнала нет.

2. Два состояния удобны, потому что их легко быстро и надёжно отличать друг от друга.

Электронной схеме проще определить «есть сигнал» или «нет сигнала», чем различать много близких состояний.

3. Нет, внутри компьютера нет настоящих цифр 0 и 1.

Это условные обозначения двух физических состояний.

4. Бит — это минимальная единица информации.

Он может принимать одно из двух значений: 0 или 1.

5. Один бит имеет два возможных состояния.

0  
1

6. Несколько битов дают больше комбинаций.

Например, два бита дают четыре комбинации:

```
00
01
10
11
```

Чем больше битов, тем больше разных значений можно закодировать.

7. Текст можно превратить в двоичные данные через числовые коды символов.

Например:

```
буква → числовой код → биты
```

8. Да, можно.

Текст, музыка и фотографии отличаются правилами кодирования, но в памяти компьютера всё равно хранятся как последовательности битов.

9. Эти основы помогают понимать, что происходит под уровнем Python.

Когда позже появятся переменные, типы данных, файлы, кодировки и память, будет проще понять, почему они работают именно так.

## Главное, что нужно запомнить

1. В основе цифрового компьютера лежат электронные элементы с двумя различимыми состояниями.
2. Эти состояния условно обозначают как 0 и 1.
3. Один 0 или 1 представляет один бит информации.
4. Последовательности битов позволяют кодировать множество различных значений.
5. Текст, изображения, музыка, видео и программы в конечном итоге представляются цифровыми данными.
6. Python скрывает большую часть нижнего уровня и позволяет работать с понятными человеку объектами.

### ! Главный вывод главы

Компьютер не «говорит на языке нулей и единиц».

Нули и единицы — это удобная математическая модель двух физических состояний электронных схем.

Комбинируя огромное количество таких простых состояний, компьютер



способен хранить данные и выполнять сложнейшие программы.

## Что дальше

Теперь мы знаем, почему компьютер использует два состояния и откуда появляются 0 и 1.

Но остаётся важный вопрос:

**как именно из последовательностей битов получаются числа, буквы, изображения и другие данные?**

В следующей главе мы разберёмся, как компьютер хранит информацию и познакомимся с битами, байтами и кодированием данных подробнее.

## 2.3 Как компьютер хранит данные

---

### ! Важное уведомление

Главный вопрос главы:

**Если компьютер знает только 0 и 1, как он хранит текст, числа, фотографии, музыку и программы?**

После предыдущей главы мы уже знаем важную вещь:

Любая информация внутри компьютера состоит только из нулей и единиц.

Но возникает следующий вопрос.

Когда вы открываете фотографию, запускаете игру или читаете текстовый файл, вы не видите последовательность вроде

010011010100101011...

Вы видите изображение, буквы, числа и кнопки.

Почему?

Потому что сами биты ничего не означают.

Смысл появляется только тогда, когда существует правило их интерпретации.

---

# Бит — самая маленькая единица информации

---

Бит может принимать только два значения.

0

или

1

Это похоже на обычный выключатель.

Выключено -> 0

Включено -> 1

Одного бита почти никогда недостаточно.

Поэтому биты объединяют в группы.

---

# Байт

---

Самая известная группа — **байт**.

Один байт состоит из восьми битов.

10100110

или

00000000

или

11111111

Всего существует

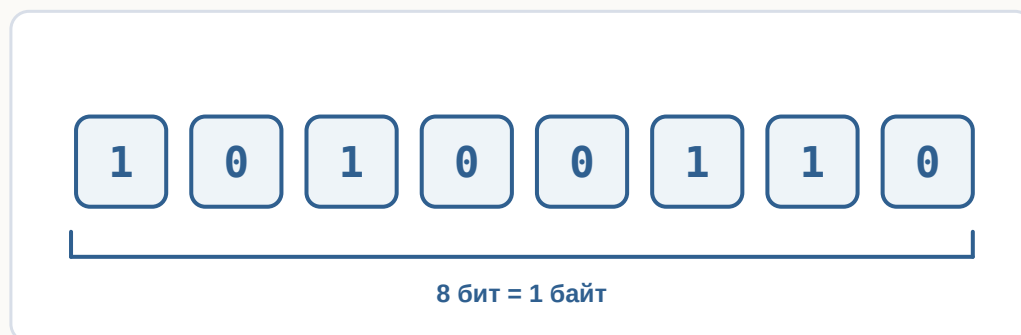
$$2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 = 256$$

различных комбинаций.

Поэтому один байт способен хранить одно из **256 различных значений**.

Это очень удобно.

Байт — это группа из восьми битов



возможных вариантов  
 **$2^8 = 256$  комбинаций**

Рисунок 1.6: Восемь битов образуют один байт.

# Как компьютер понимает, что хранится в байте?

---

Представьте число

65

Что это означает?

Без контекста — ничего.

Это может быть:

- возраст;
- скорость;
- номер страницы;
- температура.

То же самое происходит с байтами.

Последовательность

01000001

сама по себе ничего не означает.

Но если договориться считать её символом ASCII, получится буква

A

Если считать её числом —

65

То есть одинаковые биты могут означать разные вещи.

Все зависит от правил.

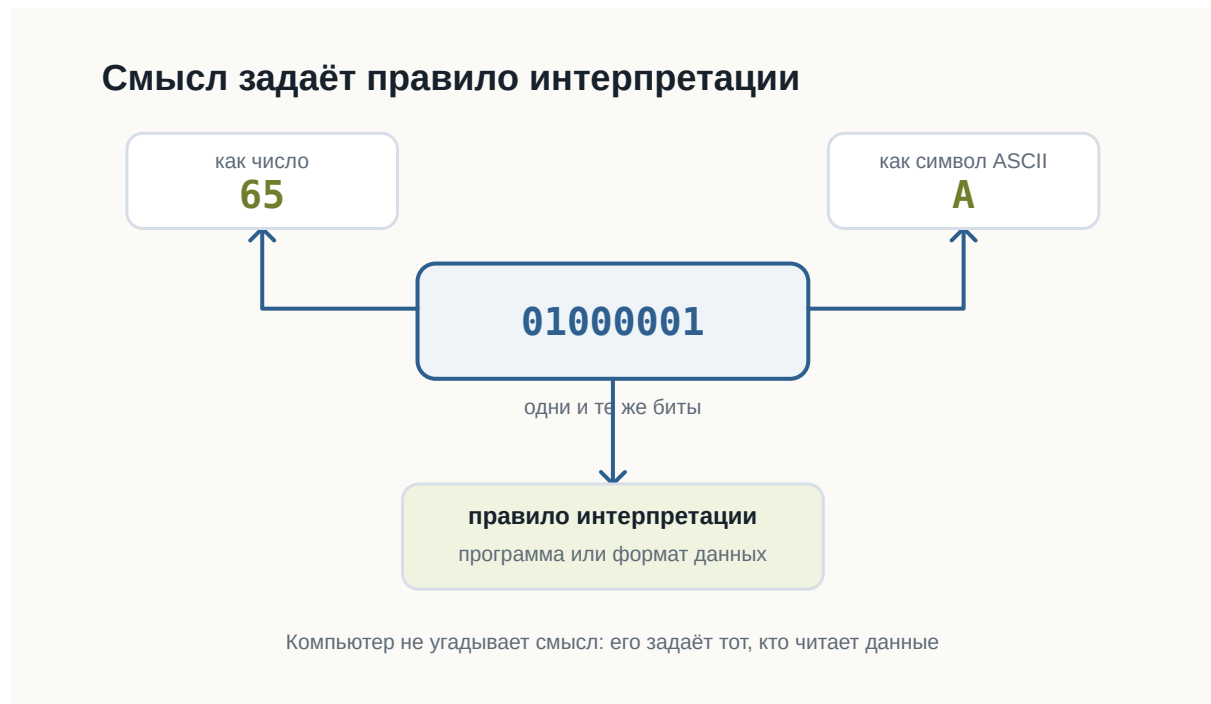


Рисунок 1.7: Одинаковые биты получают смысл только через правило интерпретации.

# Как хранится текст

---

Компьютер не хранит буквы.

Он хранит номера букв.

Например,

| Символ | Код |
|--------|-----|
| A      | 65  |
| B      | 66  |
| C      | 67  |

Когда вы набираете

ABC

в памяти оказывается примерно следующее:

65 66 67

или в двоичном виде

01000001

01000010

01000011

Позже программа снова превращает эти числа в буквы.



## Компьютер хранит не буквы, а их коды

Символ → код → биты

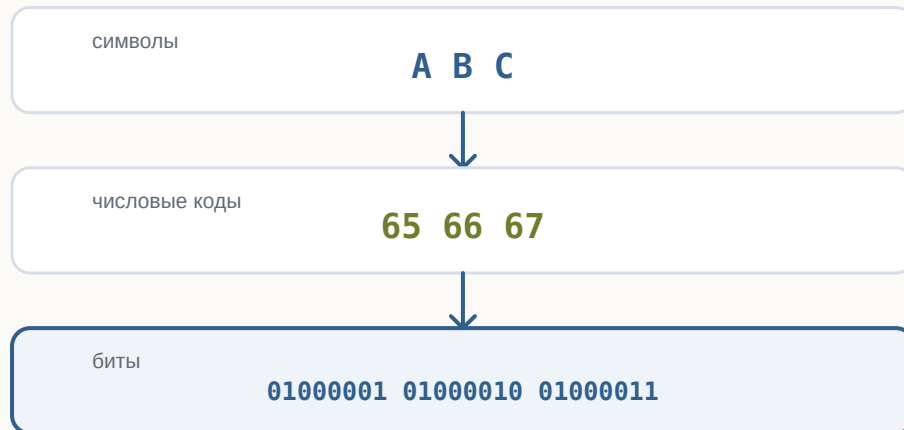


Рисунок 1.8: Текст хранится как числовые коды, представленные битами.

# Почему этого оказалось недостаточно

---

ASCII содержал только 128 символов.

Для английского языка этого хватало.

Но как быть с

Привет

□□□□□

□□□□□

□

Появился Unicode.

Он назначает номер практически каждому символу, существующему сегодня.

Поэтому современные программы способны одновременно показывать русский, японский, арабский и эмодзи.

# Как хранится число

---

Компьютер не знает, что такое “сорок два”.

Он хранит двоичное представление.

Например,

42

становится

00101010

Для компьютера это просто набор битов.

Если программа знает, что это число, она выполняет арифметику.

Если считает, что это символ, получит совершенно другой результат.

---

# Как хранится изображение

---

Фотография — это вовсе не рисунок.

Это огромная таблица пикселей.

Например,

□ □ □  
□ □ □

Каждый пиксель имеет цвет.

Цвет тоже можно записать числами.

Например,

Красный  
 $R = 255$   
 $G = 0$   
 $B = 0$

или

Зелёный

$R = 0$   
 $G = 255$   
 $B = 0$

Таким образом фотография превращается в огромное количество чисел.

А числа уже легко представить битами.

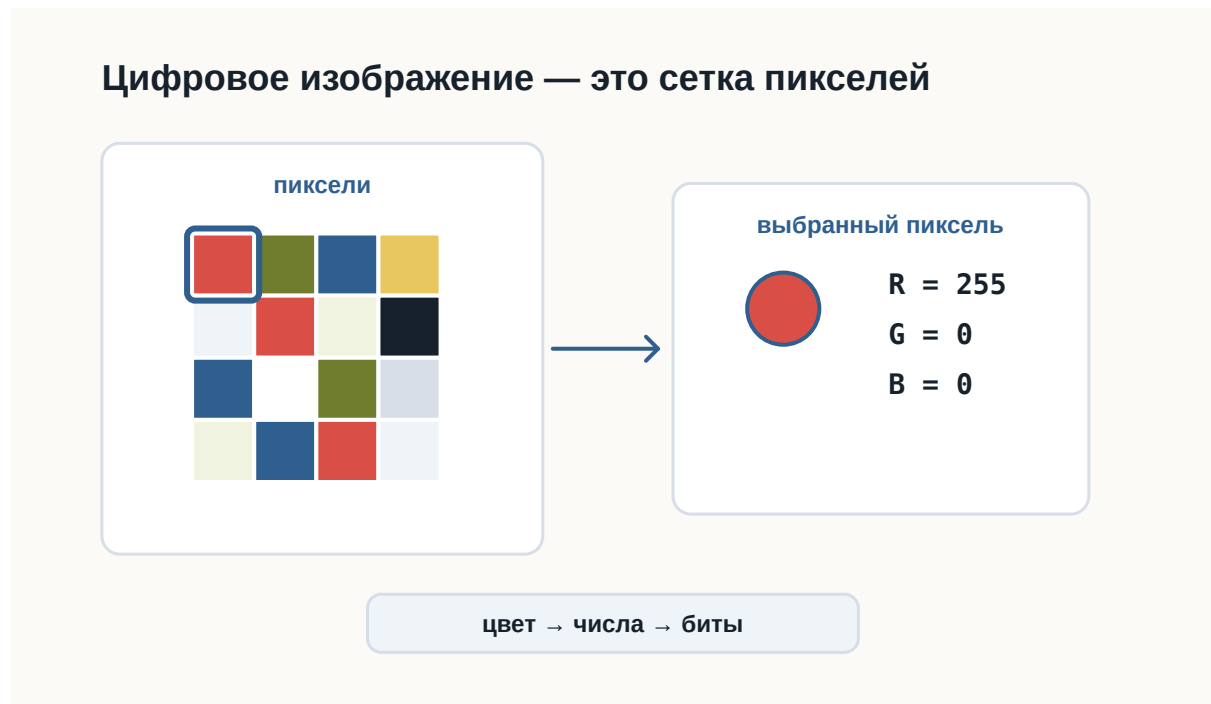


Рисунок 1.9: Изображение хранится как сетка пикселей, а цвет каждого пикселя задаётся числами RGB.

# Как хранится музыка

---

Музыка — это изменение давления воздуха.

Микрофон много тысяч раз в секунду измеряет это давление.

Получается последовательность чисел.

Например,

125

127

130

126

120

...

Компьютер хранит именно эти числа.

При воспроизведении динамик выполняет процесс в обратную сторону.

---

# Как хранится программа

---

Самое интересное — программы тоже являются данными.

Файл Python

```
print("Hello")
```

хранится как обычный текст.

Но после запуска интерпретатор читает этот текст и начинает выполнять инструкции.

То есть один и тот же набор битов может быть:

- текстом;
- программой;
- изображением;
- музыкой.

Все определяется тем, **кто именно читает эти данные.**

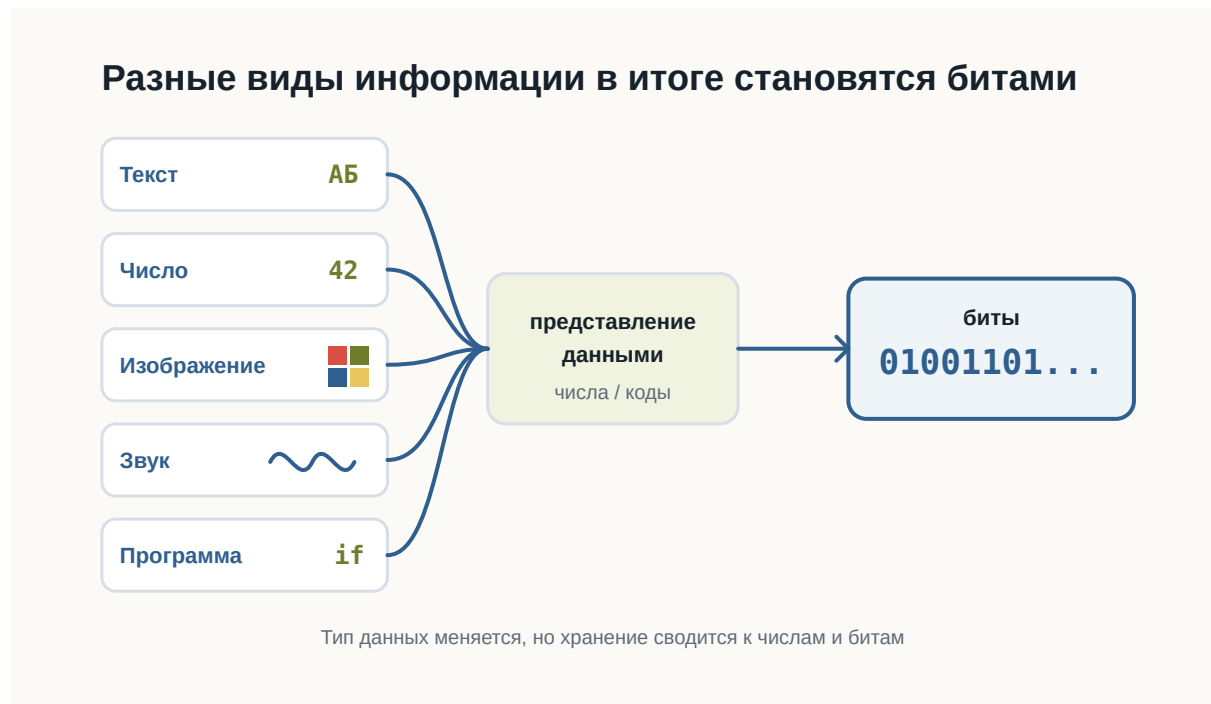


Рисунок 1.10: Текст, числа, изображения, звук и программы сводятся к числам и битам.



# Главное, что нужно запомнить

---

Компьютер хранит только последовательности битов.

Никаких букв, фотографий или музыки внутри памяти нет.

Есть лишь числа.

Именно программы знают, как превратить эти числа обратно в текст, изображения, звук или команды.

Поэтому можно сказать так:

Компьютер хранит не сами данные, а их двоичное представление.

---

# Что дальше?

---

Теперь мы понимаем:

- почему всё сводится к битам;
- как появляются байты;
- почему существуют кодировки;
- как текст, изображения и числа становятся последовательностями битов.

Но остается следующий вопрос.

Даже если программа уже хранится в памяти как набор битов, **как процессор начинает её выполнять?**

Именно об этом будет следующая глава:

**«Как компьютер выполняет программы».**

## 2.4 Как компьютер выполняет программы

---

### ! Важное уведомление

Главный вопрос главы:

**Если программа хранится в памяти как обычные данные, как процессор начинает её выполнять?**

Когда вы запускаете программу, компьютер не «читает намерения».

Он выполняет конкретную последовательность действий: находит файл, загружает нужные данные в оперативную память и передаёт инструкции процессору.

### После этой главы вы сможете

- объяснить, что происходит после запуска программы;
- понимать разницу между файлом программы и работающим процессом;
- объяснить, зачем нужна оперативная память;
- понимать общий цикл работы процессора;
- объяснить, зачем Python-коду нужен интерпретатор.

### Программа — это инструкция для компьютера

Программа может храниться как обычный файл.

Например:

```
hello.py
```

Внутри файла находится текст программы.

Сам файл ещё не выполняется. Он просто лежит на носителе как данные.

Чтобы программа начала работать, операционная система должна подготовить её к выполнению.

## Где находится программа до запуска

До запуска программа обычно находится на SSD или другом накопителе.

Накопитель подходит для долговременного хранения:

- файлы не исчезают после выключения компьютера;
- данные можно хранить долго;
- можно открыть файл позже.

Но процессор не выполняет программу прямо с накопителя.

Перед выполнением нужные данные загружаются в оперативную память.

## Что происходит после запуска

После запуска операционная система создаёт процесс.

Процесс — это работающая программа вместе с ресурсами, которые ей выделены.

Например, процесс может использовать:

- оперативную память;
- файлы;
- ввод с клавиатуры;
- вывод в терминал.



Рисунок 1.11: Путь программы от накопителя к процессору

Главная идея проста: файл программы хранится на накопителе, но во время работы программа находится в оперативной памяти, а процессор выполняет её инструкции.

## Как процессор выполняет инструкции

Процессор работает пошагово.

Он не выполняет всю программу сразу.

В упрощённом виде его работа выглядит как повторяющийся цикл:

1. получить очередную инструкцию;
2. понять, что она означает;
3. выполнить действие.

### Процессор повторяет один и тот же цикл



Рисунок 1.12: Цикл работы процессора: получить, декодировать, выполнить

Этот цикл повторяется очень быстро.

Именно поэтому программа может выполнять тысячи, миллионы или миллиарды операций за короткое время.

## Откуда процессор знает, какую инструкцию выполнять

Процессор должен знать, какую инструкцию выполнить следующей.

Для этого он хранит указатель на текущую или следующую инструкцию.

Обычно после одной инструкции процессор переходит к следующей.

Но не всегда.

Условия, циклы и вызовы функций могут изменить порядок выполнения.

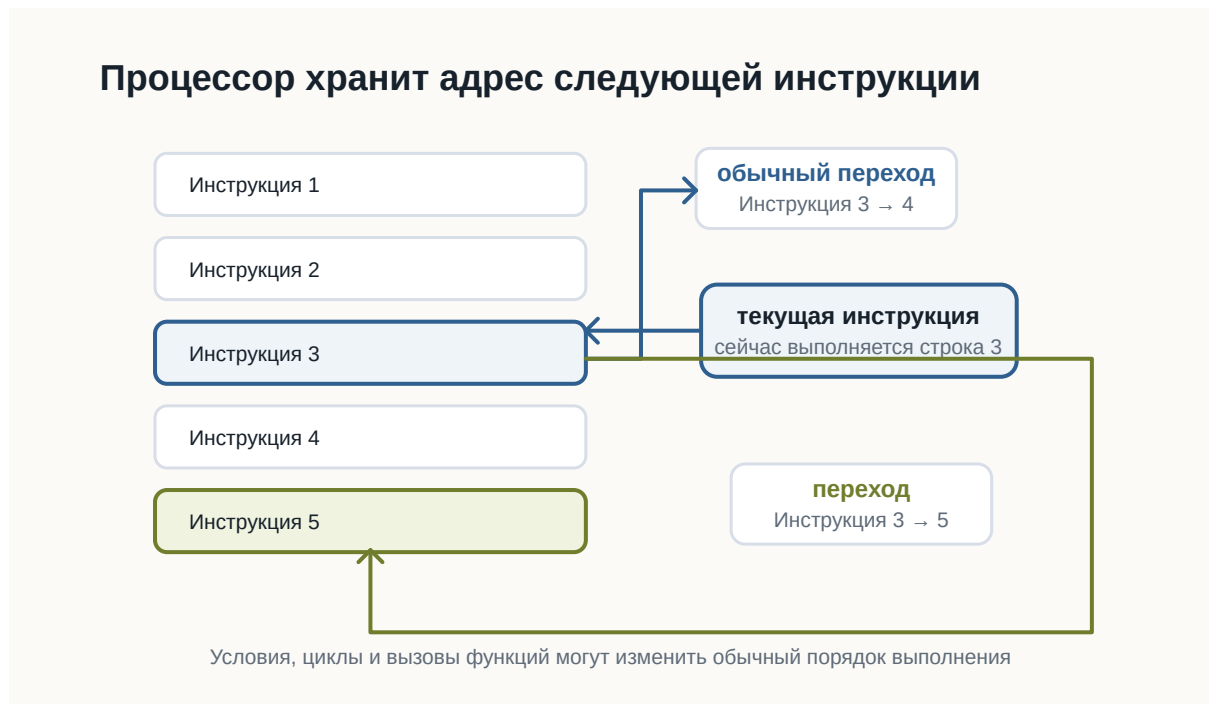


Рисунок 1.13: Указатель текущей инструкции и переходы

Это одна из причин, почему программа может не просто идти сверху вниз, а принимать решения и повторять действия.

## А где здесь Python?

Python-программа обычно хранится в файле с расширением `.py`.

Например:

```
print("Привет")
```

Процессор не выполняет такой текст напрямую.

Python-код читает специальная программа — интерпретатор Python.

Когда вы запускаете команду:

```
python hello.py
```

вы просите интерпретатор Python прочитать файл `hello.py` и выполнить инструкции внутри него.

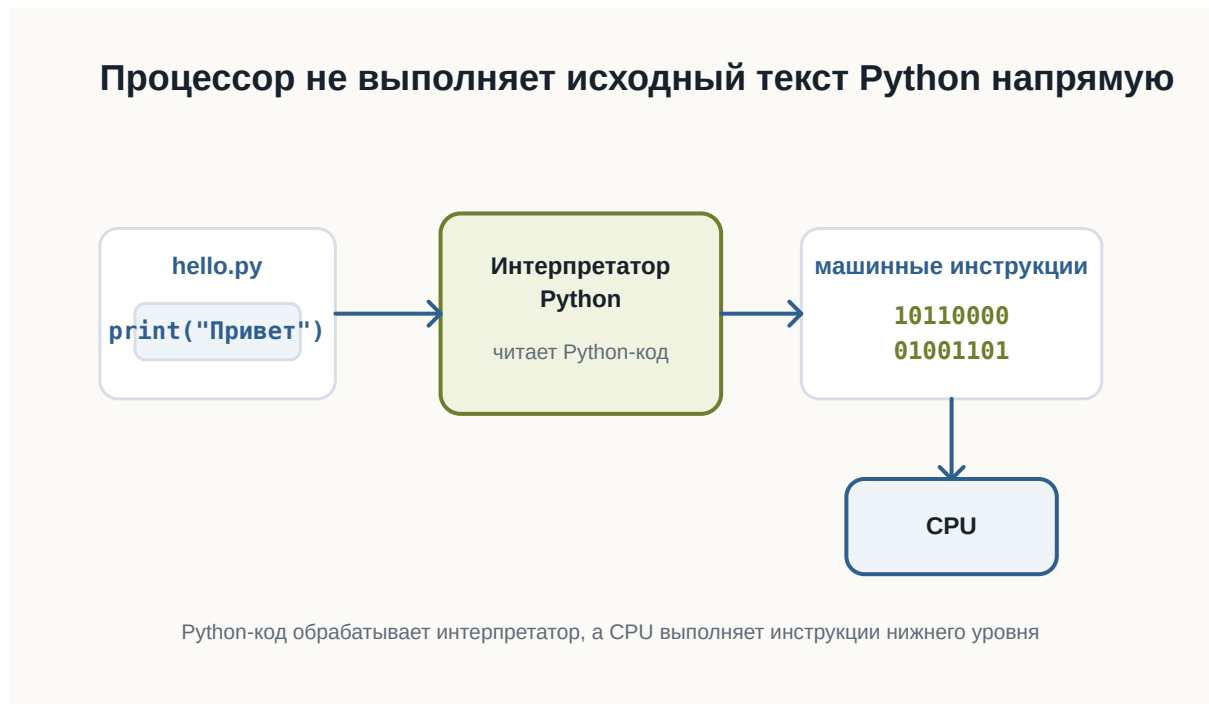


Рисунок 1.14: Роль интерпретатора Python между файлом и процессором

На этом этапе достаточно понимать общую идею: Python-код обрабатывает интерпретатор, а процессор выполняет инструкции более низкого уровня.

## Короткий итог

Программа хранится как файл.

После запуска операционная система создаёт процесс и загружает нужные данные в оперативную память.

Процессор получает инструкции, декодирует их и выполняет.

Python-код не выполняется процессором напрямую: между файлом `.py` и процессором находится интерпретатор Python.

## Практика

Объясните своими словами, чем отличается файл `hello.py` от запущенной программы.



## 2 2.5 Почему Python

---

### ! Важное уведомление

Главный вопрос главы:

**Если существуют десятки языков программирования, почему мы начинаем именно с Python?**

В предыдущих главах мы разобрались, как компьютер хранит данные и как процессор выполняет инструкции.

Теперь возникает следующий вопрос.

Если процессор в конечном счёте работает с машинными инструкциями, зачем человеку вообще нужен язык программирования?

И если языков программирования много, почему для этой книги выбран именно Python?

Чтобы ответить на этот вопрос, сначала нужно понять, зачем языки программирования появились вообще.

---

### 2.1 Можно ли писать программы сразу на машинном языке

Технически — да.

Процессор выполняет машинные инструкции.

В самом низком уровне программа действительно превращается в последовательности чисел и битов.

Условно можно представить это так:

```
10110000
01100001
```

```
11001010
00010111
```

Для процессора такой формат нормален.

Для человека — почти нет.

Попробуйте определить по этим битам:

- что делает программа;
- где начинается одна инструкция;
- где заканчивается другая;
- какое значение используется;
- что нужно изменить, чтобы программа работала иначе.

Даже небольшая программа быстро превратилась бы в огромную последовательность чисел.

Писать, читать и исправлять такой код было бы чрезвычайно сложно.



Рисунок 2.1: Человеку удобнее писать программу на языке высокого уровня

## 2.2 Язык программирования нужен человеку

Язык программирования позволяет описывать действия компьютера в форме, которую человеку намного проще читать.

Например:

```
print("Привет")
```

Эта строка гораздо понятнее, чем набор машинных инструкций.

Мы можем довольно быстро догадаться, что программа должна вывести текст.

Но процессор не понимает Python напрямую.

Поэтому между кодом программиста и процессором появляется ещё один слой.



Рисунок 2.2: Язык программирования как слой между человеком и процессором

Перевод выполняют специальные программы.

В зависимости от языка это могут быть:

- компиляторы;
- интерпретаторы;
- виртуальные машины;
- комбинации нескольких подходов.

Пока нам не нужно подробно разбираться в этих механизмах.

Важно понять главное:

Язык программирования существует прежде всего для человека, а не для процессора.

## 2.3 Почему языков программирования так много

Если один язык удобнее машинного кода, возникает естественный вопрос:

### **Почему не существует одного языка для всех программ?**

Потому что разные задачи предъявляют разные требования.

Иногда важнее:

- максимальная скорость;
- полный контроль над памятью;
- безопасность;
- простота разработки;
- работа в браузере;
- удобство обработки данных;
- быстрая автоматизация.

Поэтому разные языки делают разные компромиссы.

Например:

- C позволяет работать близко к устройству компьютера;
- JavaScript стал основным языком программирования в браузере;
- Java широко используется в крупных приложениях;
- Python делает большой акцент на читаемости и скорости разработки.

Это не означает, что один язык лучше остальных.

У каждого инструмента есть область, в которой он особенно удобен.



Рисунок 2.3: Разные языки делают разные компромиссы и удобны для разных задач

## 2.4 Почему для обучения выбран Python

Python хорошо подходит для знакомства с программированием по нескольким причинам.



Рисунок 2.4: Python выбран как удобный первый язык и профессиональный инструмент

Первая — **читаемость**.

Сравните простой фрагмент:

```
age = 20

if age >= 18:
    print("Доступ разрешён")
```

Даже не зная Python, можно приблизительно понять смысл программы.

Синтаксис не скрывает основную идею за большим количеством служебных конструкций.

---

Вторая причина — **небольшой порог входа**.

Чтобы написать простую программу, обычно не требуется сначала изучать:

- сложную систему типов;
- управление памятью;
- структуру проекта;
- процесс компиляции;
- большое количество обязательного синтаксиса.

Это позволяет быстрее перейти к самому программированию:

- переменным;
  - условиям;
  - циклам;
  - функциям;
  - структурам данных;
  - алгоритмам.
- 

Третья причина — **Python остаётся профессиональным инструментом.**

Это важно.

Мы выбираем Python не только потому, что на нём удобно учиться.

Он используется в реальных проектах.

На Python пишут:

- серверные приложения;
- инструменты автоматизации;
- тесты;
- системы обработки данных;
- научные программы;
- инструменты DevOps;
- приложения машинного обучения;
- внутренние сервисы компаний.

Поэтому знания, полученные во время обучения, не становятся бесполезными после первых глав.

---

## 2.5 Python не лучший язык для всего

Важно не создавать неправильное впечатление.

Python не является универсально лучшим языком.

Например, он может быть не лучшим выбором, когда требуется:

- писать драйвер устройства;
- создавать часть операционной системы;
- получать максимальную производительность на низком уровне;
- программировать микроконтроллер с очень ограниченной памятью;
- писать код, который должен напрямую выполняться в браузере.

В таких задачах могут лучше подходить другие языки.

Поэтому профессиональный разработчик выбирает язык не потому, что он ему нравится больше остальных.

Он выбирает инструмент под задачу.

---

## 2.6 В чём сила Python

У Python есть ещё одно важное свойство.

Сам язык относительно компактный, но вокруг него существует огромная экосистема.

Это означает, что многие сложные задачи уже имеют готовые инструменты.

Например, вместо того чтобы самостоятельно писать всё с нуля, разработчик может использовать существующие библиотеки.

Условно:

```
Python
  ↓
Библиотеки
  ↓
Готовые инструменты
  ↓
Решение задачи
```

Это позволяет сосредоточиться не на низкоуровневых деталях, а на самой задаче.

Именно поэтому Python часто используют для автоматизации и быстрого создания новых программ.

---

## 2.7 Где используется Python

Python применяется в очень разных областях.





Рисунок 2.5: Основные области применения Python

## 2.8 Веб-разработка

На Python можно создавать серверную часть сайтов и API.

Например, программа может:

- принимать запрос;
- обращаться к базе данных;
- выполнять вычисления;
- возвращать результат пользователю.

---

## 2.9 Автоматизация

Python часто используют для небольших программ и скриптов.

Например:

- переименовать тысячи файлов;
  - обработать таблицы;
  - собрать данные из нескольких источников;
  - автоматически создать отчёт;
  - проверить состояние серверов.
-

## 2.10 Анализ данных

Python широко используется для работы с большими наборами данных.

Программы могут:

- читать данные;
  - очищать их;
  - выполнять вычисления;
  - строить графики;
  - искать закономерности.
- 

## 2.11 Машинное обучение

Большая часть современной экосистемы машинного обучения доступна через Python.

При этом сложные вычисления часто выполняются не самим Python.

Python используется как удобный интерфейс к высокопроизводительным библиотекам.

---

## 2.12 Тестирование

Python также подходит для автоматического тестирования программ.

Вместо того чтобы каждый раз проверять программу вручную, можно написать другой код, который выполнит проверку автоматически.

---

## 2.13 Простота не означает примитивность

Иногда простой синтаксис создаёт впечатление, что Python — это только учебный язык.

Это не так.

Простой язык и простая задача — не одно и то же.

На Python можно написать:

```
print("Привет")
```

а можно построить сложную систему из:

- десятков сервисов;
- баз данных;
- очередей сообщений;
- фоновых процессов;
- API;
- автоматических тестов.

Сложность реальной разработки появляется не только в синтаксисе языка.

Она появляется в архитектуре, данных, взаимодействии компонентов и требованиях к системе.

---

## 2.14 Что именно мы будем изучать

Наша цель — не просто выучить команды Python.

Важно научиться мыслить как разработчик.

Поэтому дальше мы будем постепенно разбирать:

- как хранить данные в программе;
- как принимать решения;
- как повторять действия;
- как разбивать программу на функции;
- как работать со структурами данных;
- как организовывать код;
- как находить и исправлять ошибки;
- как тестировать программы.

Python будет нашим инструментом для изучения этих идей.

---

## 2.15 Что важно запомнить

Процессору не нужен Python.

Процессор выполняет машинные инструкции.

Python нужен нам, потому что писать программу на языке высокого уровня намного удобнее, чем непосредственно на машинном коде.

При этом Python:

- относительно легко читать;
- имеет небольшой порог входа;
- позволяет быстро писать программы;
- используется в реальной разработке;
- имеет большую экосистему библиотек;
- подходит для множества разных задач.

Но самое важное:

Мы изучаем не только Python. Мы изучаем программирование с помощью Python.

---

## 2.16 Что дальше?

Теперь понятно, почему для этой книги выбран Python.

Следующий вопрос уже практический.

**Что нужно установить и настроить, чтобы начать писать программы на Python?**

Этому посвящена следующая глава:

**«Рабочее пространство Python-разработчика».**

## 2.6 Рабочее пространство Python

---

Перед тем как писать программы, нужно подготовить рабочее пространство.

Рабочее пространство - это набор инструментов и папок, в которых вы будете писать, запускать и хранить код.

### После этой главы вы сможете

- понимать, зачем нужен редактор кода;
- понимать роль терминала;
- создать папку проекта;
- объяснить, зачем нужно виртуальное окружение.

### Основные инструменты

Для начала нужны:

- установленный Python;
- Visual Studio Code или другой редактор кода;
- терминал;
- браузер;
- папка проекта.

Этого достаточно, чтобы написать и запустить первую программу.



Рисунок 2.6: Основные инструменты рабочего места Python-разработчика

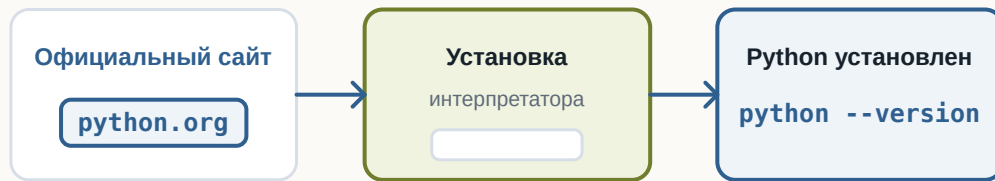
## Где скачать Python

Python скачивают с официального сайта проекта.

Важно понимать: Python устанавливается на компьютер как отдельная программа.

После установки его можно запускать из терминала.

## Python устанавливается как обычная программа



После установки команда Python становится доступна в терминале

Рисунок 2.7: Процесс установки Python

## Python — это интерпретатор

Когда вы запускаете файл .py, его читает интерпретатор Python.

Именно интерпретатор понимает Python-код и выполняет программу.

## Интерпретатор читает Python-код и выполняет его



Файл сам по себе не выполняется: его читает интерпретатор

Рисунок 2.8: Роль интерпретатора Python

## Что такое редактор кода

Редактор кода помогает писать программы, открывать файлы проекта и запускать команды.

На первом этапе достаточно понимать основные части редактора: дерево файлов, область кода и терминал.

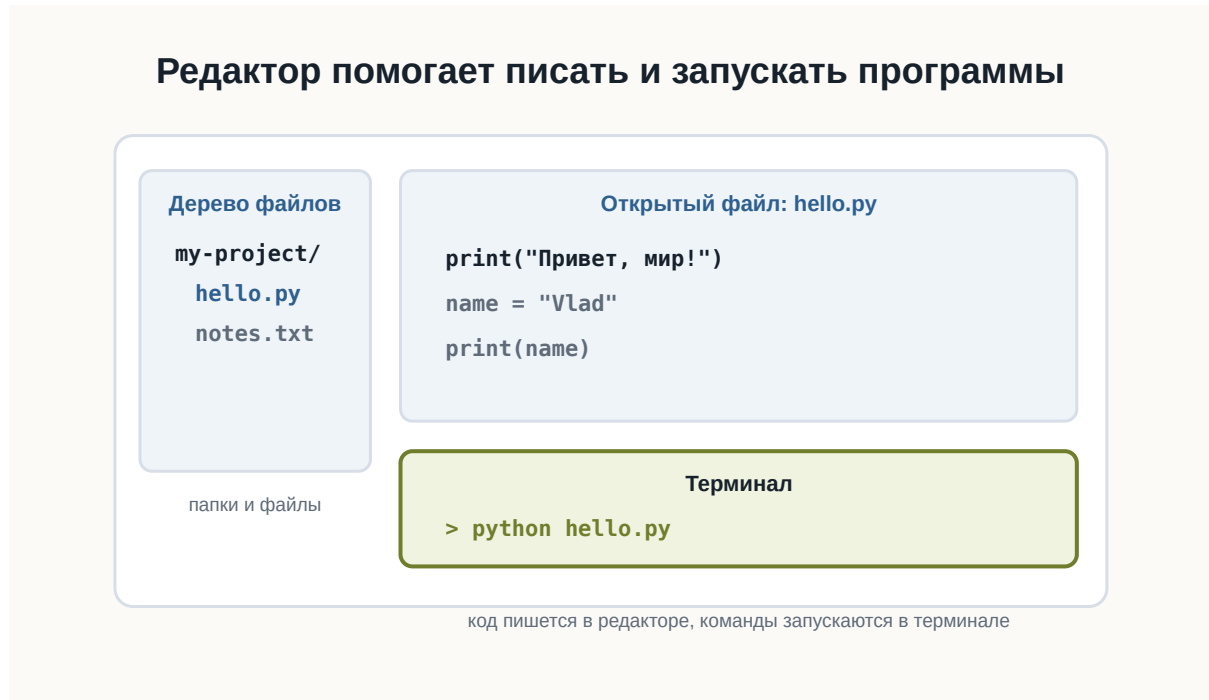


Рисунок 2.9: Условное окно редактора кода

## Структура рабочего проекта

Каждый проект лучше хранить в отдельной папке.

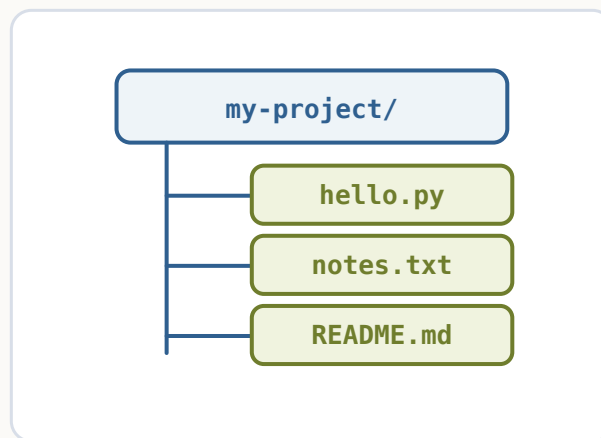
Например:

```
python-start/
```

Внутри будут лежать файлы программы, заметки, настройки и позже зависимости.



## Все файлы проекта лежат внутри одной папки



Отдельная папка помогает не смешивать файлы разных проектов

Рисунок 2.10: Простая структура Python-проекта

## Терминал

Терминал позволяет запускать команды.

Например:

```
python --version
```

Эта команда показывает установленную версию Python.

## Виртуальное окружение

Позже у разных проектов могут появиться разные зависимости.

Чтобы они не мешали друг другу, используют виртуальные окружения.

**i** Пока достаточно понимать идею

На первом шаге важно знать, что виртуальное окружение изолирует зависимости проекта. Подробно мы разберем это позже.

## Короткий итог

Рабочее пространство помогает держать код в порядке.

Редактор нужен для написания кода, терминал - для запуска команд, папка проекта - для организации файлов.

## Практика

Создайте папку `python-start` и проверьте в терминале версию Python командой `python --version`.

## 2.7 Первая программа на Python

---

### ! Важное уведомление

Главный вопрос главы:

**Как написать, запустить и понять свою первую программу на Python?**

Мы уже подготовили рабочее пространство.

Теперь можно перейти от объяснений к настоящему коду.

В этой главе мы создадим простой Python-файл, запустим его из терминала и разберём, что именно происходит на каждом этапе.

Наша первая программа будет очень маленькой.

Но путь её выполнения почти такой же, как у гораздо более сложных Python-приложений.

---

### Создаём файл программы

Создайте рабочую папку для первых экспериментов.

Например:

```
python-learning/
```

Внутри неё создайте файл:

```
hello.py
```

Расширение `.py` означает, что файл содержит исходный код на Python.

В результате структура папки будет выглядеть так:

```
python-learning/  
└─ hello.py
```

### Первая программа хранится в обычном .py-файле

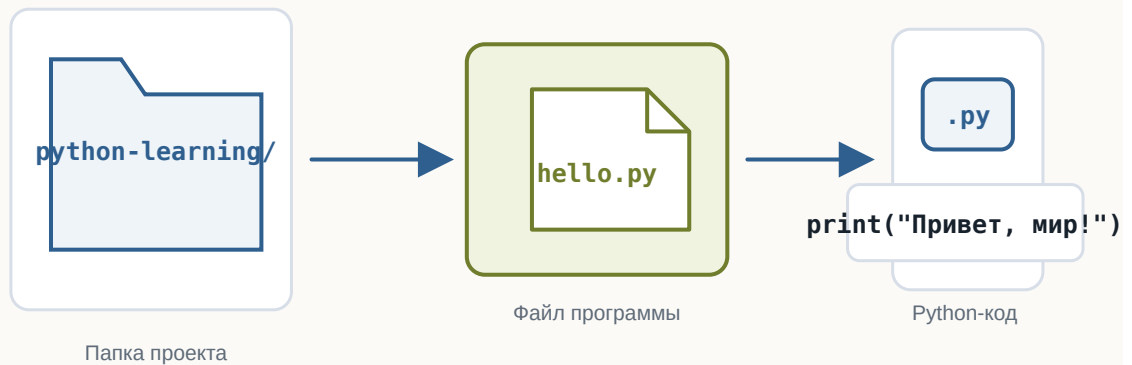


Рисунок 2.11: Первая Python-программа хранится в файле внутри проекта

Теперь откройте `hello.py` в редакторе кода.

## Пишем первую строку Python

Добавьте в файл:

```
print("Привет, мир!")
```

#### 💡 Попробуйте сами

Измените текст внутри кавычек, добавьте второй `print()` и запустите пример снова.

Сохраните изменения.

Это уже полноценная программа на Python.

Она содержит одну инструкцию.

Её задача — вывести текст на экран.

---

## Что делает print

В выражении

```
print("Привет, мир!")
```

можно выделить несколько частей.

print — это имя функции.

Круглые скобки:

```
()
```

используются для передачи функции данных.

А текст:

```
"Привет, мир!"
```

находится внутри кавычек.

Так Python понимает, что перед ним текстовое значение.

Пока не нужно подробно изучать функции и строки.

Мы разберём их позже.

Сейчас достаточно понимать:

`print(...)` просит Python вывести переданное значение.



Рисунок 2.12: Из чего состоит инструкция `print`

## Запускаем программу

Откройте терминал в папке проекта.

Убедитесь, что терминал находится в каталоге, где лежит файл:

```
hello.py
```

Теперь выполните:

```
python hello.py
```

В некоторых системах команда может называться:

```
python3 hello.py
```

Если всё работает правильно, терминал выведет:

```
Привет, мир!
```

Первая программа выполнена.

## Что произошло после команды запуска

Команда

```
python hello.py
```

содержит две важные части.

Первая:

```
python
```

указывает, какую программу нужно запустить.

Это интерпретатор Python.

Вторая:

```
hello.py
```

указывает, какой файл нужно передать интерпретатору.

Поэтому команду можно мысленно прочесть так:

```
Запусти Python и попроси его выполнить hello.py
```

После этого происходит примерно следующая последовательность:

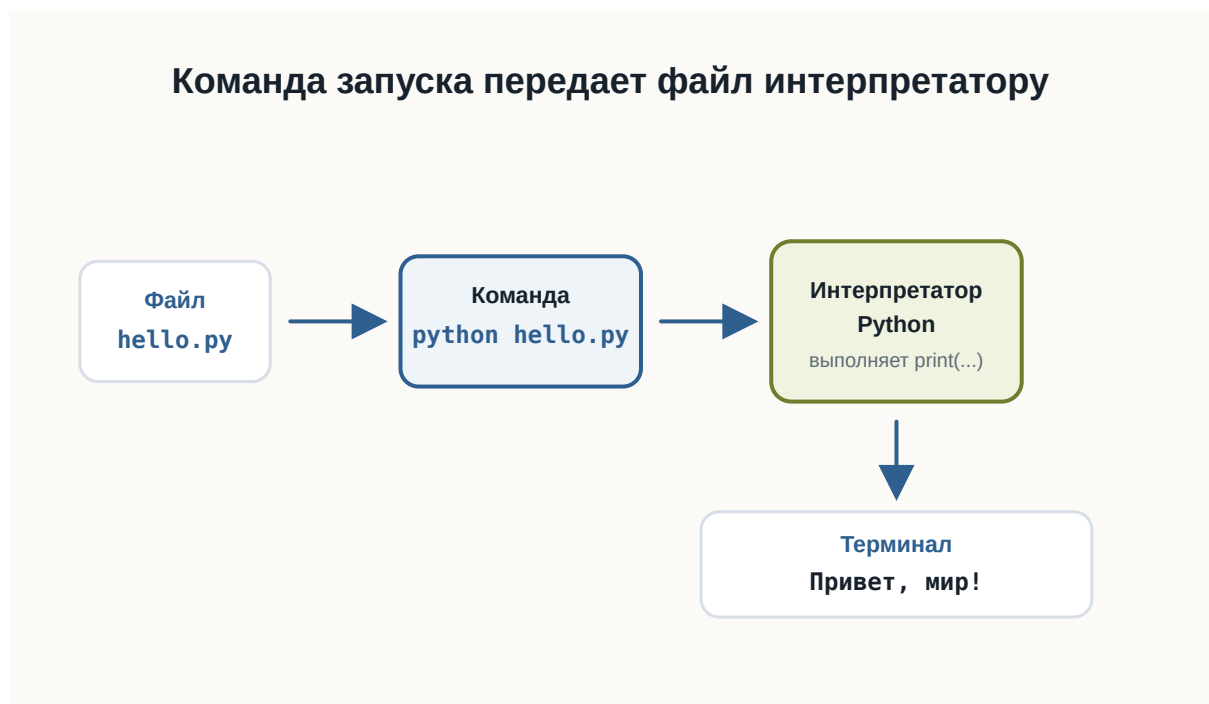


Рисунок 2.13: Путь первой Python-программы от файла до результата

---

## Код и результат — не одно и то же

Это важное различие.

Файл:

```
print("Привет, мир!")
```

содержит **код программы**.

А строка:

```
Привет, мир!
```

является **результатом выполнения программы**.

Код описывает, что компьютер должен сделать.

Результат появляется после выполнения кода.

Позже одна программа сможет:

- принимать данные;
- выполнять вычисления;
- работать с файлами;
- обращаться к сети;
- принимать решения;
- возвращать разные результаты.

Но принцип останется тем же.

---

## Добавим ещё одну инструкцию

Изменим программу:

```
print("Привет, мир!")  
print("Я изучаю Python")
```

Теперь снова запустим:



```
python hello.py
```

Результат:

```
Привет, мир!  
Я изучаю Python
```

Python выполнил инструкции сверху вниз.

Сначала первую:

```
print("Привет, мир!")
```

затем вторую:

```
print("Я изучаю Python")
```

Это один из фундаментальных принципов выполнения программ.

По умолчанию инструкции выполняются последовательно.

---

## Порядок инструкций имеет значение

Рассмотрим программу:

```
print("Первый шаг")  
print("Второй шаг")  
print("Третий шаг")
```

Результат будет:

```
Первый шаг  
Второй шаг  
Третий шаг
```

Если изменить порядок строк:

```
print("Третий шаг")  
print("Первый шаг")  
print("Второй шаг")
```

изменится и результат:

Третий шаг  
Первый шаг  
Второй шаг

Компьютер не пытается догадаться, какой порядок имел в виду программист. Он выполняет инструкции в том порядке, который указан в программе.



Рисунок 2.14: Последовательное выполнение инструкций сверху вниз

#### 💡 Попробуйте сами

Поменяйте строки местами, запустите пример снова и сравните порядок вывода.

## Программа должна быть записана точно

Попробуйте убрать одну кавычку:

```
# Ошибка сделана намеренно для учебного примера.
print("Привет, мир!)
```

### 💡 Попробуйте сами

Запустите пример с ошибкой, затем исправьте кавычку и запустите ещё раз.

Python не сможет выполнить такую программу.

Он сообщит об ошибке.

Это происходит потому, что исходный код должен соответствовать правилам языка.

Эти правила называются **синтаксисом**.

Как и в естественном языке, форма записи имеет значение.

Но компьютер гораздо строже человека.

Он не пытается догадаться, что вы хотели написать.

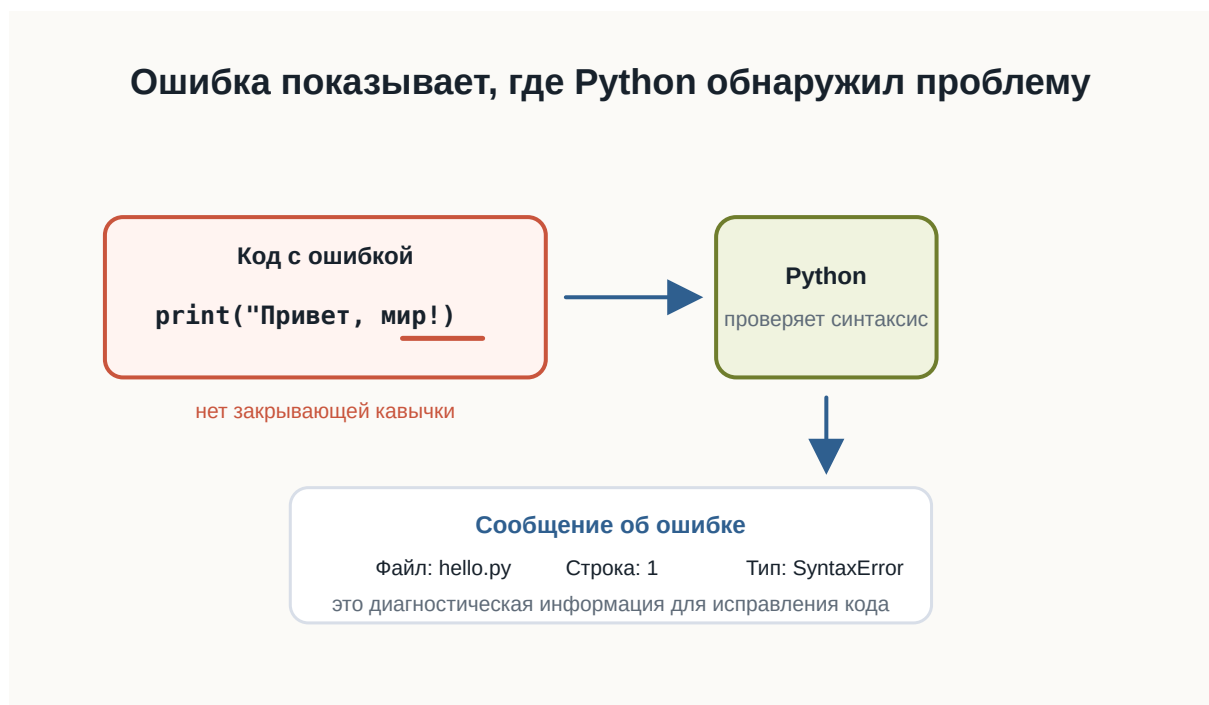


Рисунок 2.15: Синтаксическая ошибка показывает место проблемы

## Ошибка — часть программирования

Сообщение об ошибке не означает, что произошло что-то необычное.

Ошибки постоянно возникают во время разработки.

Например:

```
print("Привет")
```

Python может сообщить о синтаксической ошибке.

Это полезная информация.

Интерпретатор показывает, что программу невозможно разобрать согласно правилам языка.

Работа разработчика часто выглядит так:



Рисунок 2.16: Цикл небольших изменений, запусков и проверки результата

Этот цикл будет повторяться на протяжении всей книги.

И в профессиональной разработке происходит то же самое.

---

## Читайте сообщения об ошибках

Начинающие разработчики иногда видят красный текст в терминале и сразу считают его проблемой.

На самом деле сообщение об ошибке — это подсказка.

Например, оно может показать:

- тип ошибки;

- файл, в котором она возникла;
- номер строки;
- дополнительное описание.

Позже мы научимся читать такие сообщения подробно.

Пока важно привыкнуть к простой мысли:

Ошибка — это информация о том, почему программа не смогла выполнить ожидаемое действие.

---

## Первый маленький эксперимент

Измените текст внутри программы самостоятельно.

Например:

```
print("Меня зовут Алекс")
print("Сегодня я написал первую программу")
print("Следующая цель — понять переменные")
```

Запустите файл ещё раз.

Затем:

1. поменяйте порядок строк;
2. добавьте ещё один `print`;
3. удалите одну строку;
4. измените текст;
5. специально допустите ошибку в кавычках;
6. прочитайте сообщение Python;
7. исправьте ошибку и снова запустите программу.

Это простое упражнение важнее, чем может показаться.

Вы уже проходите основной цикл разработки:

```
написать → запустить → посмотреть → изменить → запустить снова
```

---

## Практика: программа-визитка

Создайте новый файл:

```
about_me.py
```

Напишите программу, которая выводит несколько строк о вас.

Например:

```
print("Имя: Анна")
print("Город: Казань")
print("Изучаю: Python")
print("Цель: стать разработчиком")
```

Запустите:

```
python about_me.py
```

Проверьте результат.

Затем измените программу так, чтобы она содержала не менее пяти строк вывода.

#### Совет

Не копируйте пример полностью.  
Введите собственные значения и несколько раз измените программу.  
Сейчас важнее привыкнуть самостоятельно редактировать, сохранять и запускать .py-файлы.

---

## Не бойтесь экспериментировать

Одно из преимуществ программирования — большинство экспериментов легко отменить.

Можно:

- изменить строку;
- запустить программу;
- посмотреть результат;
- вернуть код обратно;
- попробовать другой вариант.

Именно через такие небольшие эксперименты появляется понимание поведения языка.

Не ограничивайтесь только примерами из книги.

Если возникает вопрос:

А что произойдёт, если я напишу вот так?

попробуйте.

---

## Что мы уже умеем

После этой главы вы уже способны пройти полный путь небольшой программы:

```
Идея
↓
Файл .py
↓
Python-код
↓
Запуск через интерпретатор
↓
Результат в терминале
```

Вы также знаете, что:

- Python-программа хранится в .py-файле;
- `print()` позволяет вывести значение;
- интерпретатор выполняет программу;
- инструкции обычно выполняются сверху вниз;
- синтаксис должен быть записан точно;
- ошибки являются нормальной частью разработки;
- программу удобно создавать небольшими итерациями.

---

## Что важно запомнить

Первая программа:

```
print("Привет, мир!")
```

очень маленькая.

Но в ней уже присутствуют основные элементы процесса разработки:

1. исходный код;

2. файл программы;
3. интерпретатор;
4. запуск;
5. выполнение инструкции;
6. результат;
7. возможные ошибки;
8. исправление и повторный запуск.

Именно из таких простых действий постепенно строятся большие программы.

---

## Что дальше?

Мы научились запускать код.

Но пока наши программы умеют только выводить заранее записанный текст.

Следующий важный вопрос:

**Как программе запоминать значения и работать с ними?**

Для этого нам понадобится одна из основных идей программирования:

**переменные.**